

GIÁO TRÌNH PHÁT TRIỂN API CHUYÊN NGHIỆP VỚI FASTAPI (TẬP 1)

Chương 1. Giới thiệu FastAPI & Hệ sinh thái Web Hiện đại

1.1 FastAPI là gì?

FastAPI là một Web framework hiện đại, có hiệu năng cực cao dùng để xây dựng các API (Application Programming Interface) với ngôn ngữ Python. Nó được xây dựng dựa trên hai nền tảng cốt lõi là **Starlette** (cho các tính năng web toolkit) và **Pydantic** (cho việc xác thực và chuẩn hóa dữ liệu).

- **Đặc tính nổi bật:**
 - **Tốc độ (Hiệu năng):** Đạt tốc độ tương đương với NodeJS và Go nhờ kiến trúc bất đồng bộ (Asynchronous).
 - **Tăng tốc độ code:** Giảm từ 40% đến 50% các lỗi do lập trình viên viết nhờ hệ thống Python Type Hints (gợi ý kiểu dữ liệu).
 - **Tự động hóa tài liệu:** Sinh tài liệu tương tác trực quan ngay sau khi viết code mà không cần cấu hình thủ công.

1.2 ASGI là gì?

Trước đây, các framework như Django hay Flask sử dụng chuẩn **WSGI (Web Server Gateway Interface)**. WSGI hoạt động theo cơ chế đồng bộ (Synchronous), nghĩa là một luồng (thread) xử lý một request tại một thời điểm. Nếu request đó phải đợi database hoặc API bên thứ ba, luồng đó sẽ bị chặn (blocking).

ASGI (Asynchronous Server Gateway Interface) ra đời như một sự kế thừa của WSGI nhằm hỗ trợ các tính năng bất đồng bộ (**async/await**).

- ASGI cho phép xử lý hàng ngàn kết nối đồng thời trên cùng một luồng mà không bị nghẽn.
- Hỗ trợ toàn diện các giao thức hiện đại như **WebSockets**, **HTTP/2**, và **Long Polling**.

1.3 Uvicorn là gì?

FastAPI chỉ là một ứng dụng web (Web Application), bản thân nó không thể tự lắng nghe các cổng mạng (port) để tiếp nhận HTTP request từ Client. Lúc này, chúng ta cần một ASGI Server.

Uvicorn là một ASGI server chạy bằng Python có hiệu năng cực kỳ mạnh mẽ, sử dụng thư viện **uvloop** (một bộ thực thi vòng lặp sự kiện viết bằng C). Uvicorn đóng vai trò là động cơ đứng ở cổng mạng, tiếp nhận request và chuyển tiếp vào cho ứng dụng FastAPI xử lý, sau đó nhận kết quả và trả về cho người dùng.

1.4 RESTful API

REST (Representational State Transfer) là một phong cách kiến trúc thiết kế hệ thống phân tán. Một API tuân thủ các nguyên tắc của REST được gọi là RESTful API. Hệ thống tổ chức dựa trên các tài nguyên (Resources) được định danh bằng các URI (Uniform Resource Identifier).

Các phương thức HTTP (HTTP Methods) ánh xạ trực tiếp với các thao tác dữ liệu:

- **GET**: Đọc/Lấy tài nguyên.
- **POST**: Tạo mới tài nguyên.
- **PUT/PATCH**: Cập nhật tài nguyên.
- **DELETE**: Xóa tài nguyên.

1.5 JSON (JavaScript Object Notation)

JSON là định dạng trao đổi dữ liệu tiêu chuẩn gọn nhẹ, dễ đọc đối với con người và dễ phân tích đối với máy tính. Ví dụ một chuỗi JSON biểu diễn một tài khoản người dùng:

JSON

```
{  
  "id": 1,  
  "username": "anhduong",  
  "email": "duong.cao@example.com",  
  "is_active": true  
}
```

1.6 HTTP Request/Response

Mọi giao tiếp trên Web đều tuân theo mô hình Yêu cầu - Phản hồi qua giao thức HTTP:

- **HTTP Request**: Bao gồm URL, Method, Headers (Metadata chứa token, loại dữ liệu), và Body (Dữ liệu gửi lên).
- **HTTP Response**: Bao gồm Status Code (Mã trạng thái như 200 OK, 404 Not Found, 500 Error), Headers, và Body (Dữ liệu trả về, thường là JSON).

1.7 Tự động hóa tài liệu với Swagger (/docs và /redoc)

Nhờ việc định nghĩa kiểu dữ liệu chặt chẽ, FastAPI tự động phân tích mã nguồn và xuất ra đặc tả OpenAPI. Từ đặc tả này, hệ thống cung cấp sẵn hai giao diện:

- **/docs** (Swagger UI): Cho phép các lập trình viên Front-end xem cấu trúc API và tương tác thử nghiệm (nhập tham số, bấm nút Gửi để xem kết quả trực tiếp).
- **/redoc** (ReDoc): Định dạng tài liệu phẳng, sạch sẽ, tối ưu cho việc đọc và tra cứu tài liệu kỹ thuật chuyên sâu.

Chương 2. Cấu trúc dự án chuẩn Enterprise

2.1 Luồng đi của dữ liệu (Data Flow Diagram)

Để một hệ thống backend hoạt động ổn định, dễ bảo trì và mở rộng khi quy mô dự án lên tới hàng trăm nghìn dòng code, dữ liệu từ Client gửi lên phải đi qua một lộ trình phân lớp nghiêm ngặt:

Plaintext

Client (Trình duyệt/Mobile App)

|

▼ [Gửi HTTP Request kèm dữ liệu JSON]

Router (Endpoint tiếp nhận, kiểm tra định dạng dữ liệu qua Pydantic Schema)

|

▼ [Chuyển dữ liệu sạch sang tầng xử lý]

Service (Nơi xử lý Logic doanh nghiệp: tính toán, kiểm tra quyền, liên kết nhiều thực thể)

|

▼ [Gọi các hàm thao tác DB thuần túy]

CRUD (Thực thi các câu lệnh thêm, sửa, xóa với Database)

|

▼ [Ánh xạ dữ liệu thông qua Class Python]

Model (SQLAlchemy Mapping đại diện cho bảng vật lý)

|

▼ [Thực thi ghi xuống ổ đĩa]

Database (Hệ quản trị CSDL như MySQL, PostgreSQL)

2.2 Chi tiết cấu trúc cây thư mục tiêu chuẩn

Dưới đây là sơ đồ tổ chức mã nguồn chuẩn doanh nghiệp. Các file được chia tách theo nguyên lý Đơn nhiệm (Single Responsibility Principle):

Plaintext

app/

|

|— main.py # Điểm khởi tạo ứng dụng, cấu hình Middleware, CORS, và Router tổng

|— database.py # Cấu hình khởi tạo Engine, SessionLocal quản lý transaction kết nối

DB

|— config.py # Đọc và quản lý biến môi trường (.env) bằng Pydantic Settings

|— models/ # Thư mục chứa các tệp định nghĩa cấu trúc bảng Database

(SQLAlchemy Models)

| |— __init__.py

| |— user.py

| |— product.py

|— schemas/ # Thư mục chứa cấu trúc định dạng dữ liệu đầu vào/đầu ra (Pydantic

Schemas)

| |— __init__.py

| |— user.py

| |— product.py

|— crud/ # Thư mục chứa các truy vấn DB thuần túy, tái sử dụng cao

| |— __init__.py

| |— user.py

```

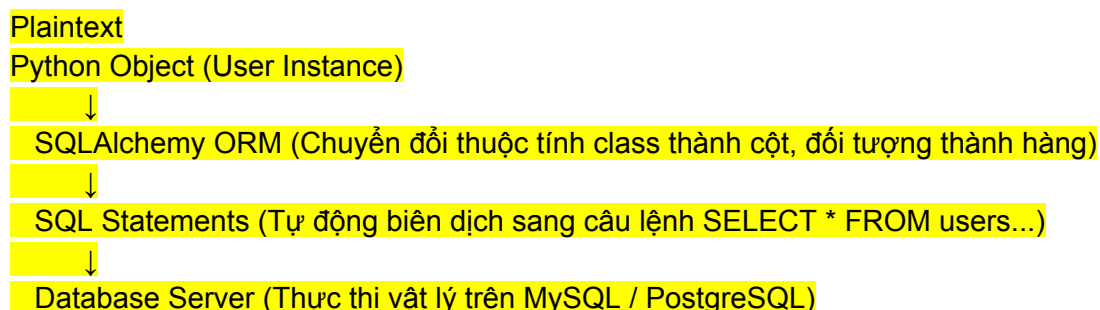
|   └─ product.py
|   └─ routers/      # Thư mục phân chia luồng endpoint theo phân hệ chức năng
|       └─ __init__.py
|       └─ auth.py
|       └─ users.py
|       └─ products.py
|   └─ services/     # Thư mục xử lý nghiệp vụ phức tạp liên quan đến nhiều bảng hoặc bên
thứ 3
|       └─ __init__.py
|       └─ payment.py
|   └─ utils/        # Các hàm hỗ trợ dùng chung (băm mật khẩu, gửi email, định dạng ngày
tháng)
|   └─ exceptions/   # Định nghĩa các lỗi tùy biến (Custom Exception) của hệ thống
|   └─ middleware/    # Các bộ lọc tiền tiên xử lý Request và hậu xử lý Response (Log, mã
hóa)

```

Chương 3. Tối ưu hóa SQLAlchemy ORM

3.1 Mô hình chuyển đổi dữ liệu (Mapping Flow)

SQLAlchemy hoạt động như một cầu nối thông minh, giúp lập trình viên thao tác với cơ sở dữ liệu bằng tư duy lập trình hướng đối tượng (OOP) thay vì viết các câu lệnh SQL thô bằng chuỗi string.



3.2 Các thành phần cấu thành SQLAlchemy

- **Engine:** Trái tim của kết nối. Nó chịu trách nhiệm giữ các cổng kết nối vật lý tới DB Server và quản lý cơ chế Connection Pooling (tái sử dụng các kết nối để tăng tốc hệ thống, tránh việc tạo mới kết nối liên tục gây thất nút cổ chai).
- **Session:** Đại diện cho một phiên làm việc (Transaction). Mọi thao tác thêm, sửa, xóa dữ liệu đều nằm trong một Session. Nếu có lỗi xảy ra, Session cho phép hủy bỏ toàn bộ thao tác (**rollback**), nếu thành công sẽ lưu vĩnh viễn (**commit**).
- **Base (Declarative Base):** Một lớp cơ sở cha. Khi một Class Python kế thừa từ lớp **Base** này, SQLAlchemy sẽ hiểu đây là một thực thể đại diện cho một bảng trong Database.

- **ORM Mapping:** Quá trình ánh xạ: Class tương ứng với Table, Thuộc tính (Attribute) tương ứng với Column, và một Đối tượng (Instance) cụ thể tương ứng với một Dòng (Row) trong bảng dữ liệu.

3.3 Hướng dẫn thiết lập mã nguồn thực tế kết nối DB (**app/database.py**)

Python

```
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker
```

1. Đường dẫn kết nối tới cơ sở dữ liệu MySQL

```
DATABASE_URL = "mysql+pymysql://root:password@localhost:3306/enterprise_db"
```

2. Khởi tạo Engine với cấu hình Pool tối ưu

```
engine = create_engine(
    DATABASE_URL,
    pool_size=20,      # Giữ tối đa 20 kết nối mở sẵn trong pool
    max_overflow=10,    # Cho phép mở thêm tối đa 10 kết nối nếu quá tải
    pool_recycle=3600   # Tự động làm mới kết nối sau 1 giờ để tránh lỗi mất kết nối từ
MySQL
)
```

3. Khởi tạo nhà máy sản xuất Session quản lý phiên làm việc

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

4. Tạo lớp Base để các Model kế thừa

```
Base = declarative_base()
```

3.4 Định nghĩa các mối quan hệ (Relationships) bằng mã nguồn thực tế

Mối quan hệ Một - Nhiều (One To Many) & Khóa ngoại (ForeignKey)

Một Người dùng (**User**) có thể tạo ra Nhiều Bài viết (**Post**).

Python

```
# app/models/user.py
from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import relationship
from app.database import Base
```

```
class User(Base):
    __tablename__ = "users"
```

```
id = Column(Integer, primary_key=True, index=True)
username = Column(String(50), unique=True, nullable=False)
email = Column(String(100), unique=True, index=True)
```

```
# Thiết lập mối quan hệ: trở tới Class 'Post' và liên kết ngược qua thuộc tính 'author'
```

```
posts = relationship("Post", back_populates="author", cascade="all, delete-orphan")
```

```
# app/models/post.py
```

```
from sqlalchemy import Column, Integer, String, ForeignKey
```

```
from sqlalchemy.orm import relationship
```

```
from app.database import Base
```

```
class Post(Base):
```

```
    __tablename__ = "posts"
```

```
    id = Column(Integer, primary_key=True, index=True)
```

```
    title = Column(String(200), nullable=False)
```

```
    content = Column(String(1000))
```

```
    user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"), nullable=False)
```

```
    # Thiết lập liên kết ngược lại với thực thể User
```

```
    author = relationship("User", back_populates="posts")
```

Mối quan hệ Một - Một (One To One)

Một Người dùng (**User**) chỉ có duy nhất Một Hồ sơ chi tiết (**Profile**). Định nghĩa tương tự Một-Nhiều nhưng thêm tham số `uselist=False` ở lớp cha.

```
Python
```

```
# Thêm thuộc tính này vào trong Class User ở trên:
```

```
profile = relationship("Profile", back_populates="user", uselist=False, cascade="all, delete-orphan")
```

```
# Định nghĩa bảng Profile độc lập:
```

```
class Profile(Base):
```

```
    __tablename__ = "profiles"
```

```
    id = Column(Integer, primary_key=True, index=True)
```

```
    avatar_url = Column(String(255))
```

```
    bio = Column(String(500))
```

```
    user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"), unique=True)
```

```
    user = relationship("User", back_populates="profile")
```

Mối quan hệ Nhiều - Nhiều (Many To Many)

Một Sản phẩm (**Product**) có thể thuộc về Nhiều Danh mục (**Category**), và ngược lại một Danh mục có thể chứa Nhiều Sản phẩm. Ta cần một bảng trung gian (Association Table).

```
Python
```

```

from sqlalchemy import Table, Column, Integer, ForeignKey
from app.database import Base

# Bảng trung gian thuần túy chứa khóa ngoại của cả hai bảng chính
product_category_association = Table(
    "product_category",
    Base.metadata,
    Column("product_id", Integer, ForeignKey("products.id", ondelete="CASCADE"),
    primary_key=True),
    Column("category_id", Integer, ForeignKey("categories.id", ondelete="CASCADE"),
    primary_key=True)
)

class Product(Base):
    __tablename__ = "products"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)

# Kết nối Nhiều - Nhiều thông qua tham số secondary
categories = relationship("Category", secondary=product_category_association,
back_populates="products")

class Category(Base):
    __tablename__ = "categories"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)

    products = relationship("Product", secondary=product_category_association,
back_populates="categories")

```

Chương 4. Xác thực và chuẩn hóa dữ liệu với Pydantic v2

4.1 BaseModel

- **Khái niệm:** Lớp cơ sở của Pydantic dùng để định nghĩa cấu trúc dữ liệu (Data Schema). Nó tự động chuyển đổi kiểu dữ liệu (Data Parsing) và kiểm tra tính hợp lệ khi dữ liệu đi vào hệ thống.
- **Mã nguồn ví dụ:**

```

Python
from pydantic import BaseModel

class UserCreate(BaseModel):
    username: str
    email: str
    age: int

```

- **Giải thích từng dòng:**
 - `class UserCreate(BaseModel):` Khai báo lớp Schema kế thừa từ `BaseModel`.
 - `username: str:` Ràng buộc trường `username` phải là chuỗi ký tự.
 - `age: int:` Nếu Client gửi lên dạng chuỗi `"25"`, Pydantic sẽ tự chuyển thành số nguyên `25`.
- **Lỗi thường gặp:** `ValidationError` xảy ra khi Client truyền `age: "ba mươi"` (không thể ép kiểu sang số nguyên).
- **Best Practice:** Luôn tách biệt Schema tạo mới (`UserCreate`) và Schema phản hồi (`UserOut`).

4.2 Field()

- **Khái niệm:** Hàm dùng để cấu hình nâng cao cho từng thuộc tính trong Schema (thiết lập giá trị mặc định, độ dài chuỗi, mô tả tài liệu Swagger).
- **Mã nguồn ví dụ:**

Python

```
from pydantic import BaseModel, Field
```

```
class ProductCreate(BaseModel):
```

```
    name: str = Field(..., min_length=3, max_length=50, description="Tên sản phẩm")
```

```
    price: float = Field(gt=0, description="Giá phải lớn hơn 0")
```

- **Giải thích từng dòng:**
 - `Field(...)`: Dấu ba chấm đại diện cho trường bắt buộc (Required).
 - `min_length=3`: Bắt buộc chuỗi phải từ 3 ký tự trở lên.
 - `gt=0`: Viết tắt của "Greater Than 0", ngăn chặn giá trị âm hoặc bằng 0.

4.3 Optional & List

- **Khái niệm:** `Optional` (hoặc toán tử `| None` trong Python 3.10+) biểu thị trường dữ liệu có thể bị khuyết. `List` biểu thị một mảng dữ liệu.
- **Mã nguồn ví dụ:**

Python

```
from pydantic import BaseModel
```

```
from typing import Optional, List
```

```
class OrderSchema(BaseModel):
```

```
    note: Optional[str] = None
```

```
    items: List[int] = []
```

- **Giải thích từng dòng:**

- `note: Optional[str] = None`: Trường ghi chú không bắt buộc, mặc định là `None`.
- `items: List[int] = []`: Danh sách các mã ID sản phẩm, mặc định là mảng rỗng nếu không truyền.

4.4 Enum

- **Khái niệm:** Định nghĩa một tập hợp các hằng số cố định, ngăn chặn Client truyền dữ liệu nằm ngoài danh mục cho phép.
- **Mã nguồn ví dụ:**

Python

```
from enum import Enum
```

```
from pydantic import BaseModel
```

```
class StatusEnum(str, Enum):
```

```
    pending = "pending"
```

```
    active = "active"
```

```
class UserUpdate(BaseModel):
```

```
    status: StatusEnum
```

- **Giải thích từng dòng:**

- `class StatusEnum(str, Enum)`: Tạo một Enum kế thừa kiểu `str` để tương thích tốt với chuỗi JSON.

- Nếu Client truyền "blocked", hệ thống lập tức chặn lại và báo lỗi lỗi định dạng dữ liệu (Mã 422).

4.5 model_dump() & from_attributes=True

- **Khái niệm:** * `model_dump()`: Chuyển đổi đối tượng Pydantic thành một Dict thuần túy của Python (thay thế cho `dict()` ở Pydantic v1).
 - `from_attributes=True`: Cấu hình cho phép Pydantic đọc dữ liệu trực tiếp từ các đối tượng ORM Model của SQLAlchemy (thay thế cho `orm_mode = True`).
- **Mã nguồn ví dụ:**

Python

```
class UserOut(BaseModel):
```

```
    id: int
```

```
    username: str
```

```
class Config:
```

```
    from_attributes = True
```

```
# Sử dụng:
```

```
# user_dict = user_schema.model_dump()
```

Chương 5. Quy trình xử lý CRUD toàn tập

5.1 Sơ đồ chu kỳ vòng đời dữ liệu (Data Lifecycle)

Plaintext

Request (Client gửi JSON)



Schema Validation (Pydantic kiểm tra cấu trúc và kiểu dữ liệu)



Model Transform (Chuyển đổi dữ liệu sạch sang SQLAlchemy Object)



Database Engine (Thực thi truy vấn SQL vật lý xuống MySQL)



Response Serialization (SQLAlchemy Object đổi thành Pydantic Output → JSON trả về Client)

5.2 Mã nguồn triển khai lớp CRUD mẫu (`app/crud/user.py`)

Python

```
from sqlalchemy.orm import Session
```

```
from app.models.user import User
```

```
from app.schemas.user import UserCreate, UserUpdate
```

1. CREATE

```
def create_user(db: Session, user_in: UserCreate):
```

```
    db_user = User(
```

```
        username=user_in.username,
```

```
        email=user_in.email,
```

```
        hashed_password="hashed_" + user_in.password # Giả định mã hóa đơn giản
```

```
    )
```

```
    db.add(db_user)
```

```
    db.commit() # Ghi vĩnh viễn dữ liệu xuống MySQL
```

```
db.refresh(db_user) # Cập nhật lại các trường tự động sinh như ID, Created_at
```

```
return db_user
```

2. READ (Lấy chi tiết)

```
def get_user_by_id(db: Session, user_id: int):
```

```
    return db.query(User).filter(User.id == user_id).first()
```

3. UPDATE

```
def update_user(db: Session, user_id: int, user_in: UserUpdate):
```

```
    db_user = db.query(User).filter(User.id == user_id).first()
```

```
    if db_user:
```

```
        update_data = user_in.model_dump(exclude_unset=True) # Chỉ lấy các trường Client  
        thực tế truyền lên
```

```
        for key, value in update_data.items():
```

```
            setattr(db_user, key, value)
```

```
        db.commit()
```

```
        db.refresh(db_user)
```

```
    return db_user
```

4. DELETE

```
def delete_user(db: Session, user_id: int):
```

```
    db_user = db.query(User).filter(User.id == user_id).first()
```

```
    if db_user:
```

```
        db.delete(db_user)
```

```
        db.commit()
```

```
return db_user
```

Chương 6. Tra cứu chuyên sâu phương thức Query SQLAlchemy

Chương này cung cấp hướng dẫn chi tiết cho các hàm truy vấn cốt lõi. Mỗi hàm gồm: **Khái niệm** → **Mã Python** → **SQL tương đương**.

6.1 query() & filter() & filter_by()

- **Khái niệm:** `query()` chỉ định bảng cần lấy dữ liệu; `filter()` dùng toán tử logic Python (như `==`, `>`); `filter_by()` dùng tham số đặt tên (keyword arguments).
- **Mã nguồn ví dụ:**

Python

```
# Cách 1: Sử dụng filter()
```

```
res1 = db.query(Product).filter(Product.price > 100, Product.status == "active").all()
```

```
# Cách 2: Sử dụng filter_by()
```

```
res2 = db.query(Product).filter_by(status="active").all()
```

- **SQL tương đương:**

SQL

```
SELECT * FROM products WHERE price > 100 AND status = 'active';
```

6.2 all() & first() & one() & one_or_none()

- **Khái niệm:** * `all()`: Lấy toàn bộ kết quả thành một danh sách (Mảng).
 - `first()`: Lấy bản ghi đầu tiên, nếu không có trả về `None`.
 - `one()`: Bắt buộc trả về đúng 1 bản ghi. Lỗi `NoResultFound` nếu có 0 bản ghi, lỗi `MultipleResultsFound` nếu > 1 bản ghi.
 - `one_or_none()`: Giống `one()` nhưng nếu có 0 bản ghi thì trả về `None`.

6.3 limit() & offset() (Phân trang - Pagination)

- **Khái niệm:** `limit()` giới hạn số lượng dòng lấy ra; `offset()` bỏ qua số dòng ban đầu (dùng để chuyển trang).
- **Mã nguồn ví dụ:**

Python

Lấy trang số 3, mỗi trang 10 sản phẩm

```
res = db.query(Product).limit(10).offset(20).all()
```

- **SQL tương đương:**

SQL

```
SELECT * FROM products LIMIT 10 OFFSET 20;
```

6.4 join() & outerjoin()

- **Khái niệm:** `join()` thực hiện Inner Join bảng liên kết; `outerjoin()` thực hiện Left Outer Join (lấy toàn bộ bảng bên trái bất kể bảng bên phải có dữ liệu hay không).
- **Mã nguồn ví dụ:**

Python

```
res = db.query(User).join(Post).filter(Post.title.like("%FastAPI%")).all()
```

- **SQL tương đương:**

SQL

```
SELECT users.* FROM users INNER JOIN posts ON users.id = posts.user_id WHERE posts.title LIKE '%FastAPI%';
```

6.5 order_by() & group_by() & count()

- **Khái niệm:** `order_by()` sắp xếp thứ tự dữ liệu; `group_by()` gom nhóm các hàng dữ liệu; `count()` đếm tổng số bản ghi.
- **Mã nguồn ví dụ:**

Python

```
from sqlalchemy import func
```

```
# Sắp xếp sản phẩm giảm dần theo giá
```

```
products_desc = db.query(Product).order_by(Product.price.desc()).all()
```

```
# Đếm số lượng sản phẩm theo từng danh mục
```

```
category_counts = db.query(Product.category_id,  
func.count(Product.id)).group_by(Product.category_id).all()
```

- **SQL tương đương:**

SQL

```
SELECT * FROM products ORDER BY price DESC;
```

```
SELECT category_id, count(id) FROM products GROUP BY category_id;
```

6.6 matching toán tử mở rộng (like, ilike, in_, between)

- **Khái niệm:** Các toán tử lọc chuỗi và khoảng giá trị nâng cao.
- **Mã nguồn ví dụ:**

Python

```
# Lọc sản phẩm có giá từ 50 đến 150 và thuộc nhóm id 1, 2, 3
```

```
res = db.query(Product).filter(  
  
    Product.price.between(50, 150),  
  
    Product.category_id.in_([1, 2, 3]),  
  
    Product.name.like("%điện thoại%")  
  
).all()
```

- **SQL tương đương:**

SQL

```
SELECT * FROM products
```

```
WHERE price BETWEEN 50 AND 150
```

```
AND category_id IN (1, 2, 3)
```

```
AND name LIKE '%điện thoại%';
```

(Lưu ý: *ilike* dùng cho tìm kiếm không phân biệt chữ hoa/chữ thường trong các DB có hỗ trợ).

Chương 7. Tối ưu hóa HTTP Response và Serialization

7.1 response_model & status_code

- **Khái niệm:** `response_model` định nghĩa cấu trúc Schema Pydantic đầu ra tại route decorator. Nó đóng vai trò như một bộ lọc (Filter): tự động loại bỏ các trường nhạy cảm (như `hashed_password`) và chuẩn hóa kiểu dữ liệu trước khi trả về Client. `status_code` xác định mã trạng thái HTTP mặc định khi API thành công.
- **Mã nguồn ví dụ:**

Python

```
from fastapi import APIRouter, Depends, status
from sqlalchemy.orm import Session
from app.schemas.user import UserOut
from app.database import get_db
from app.crud import user as user_crud
```

```
router = APIRouter()
```

```
@router.get("/users/{user_id}", response_model=UserOut,
status_code=status.HTTP_200_OK)
def read_user(user_id: int, db: Session = Depends(get_db)):
    db_user = user_crud.get_user_by_id(db, user_id=user_id)
    return db_user
```

- **Giải thích từng dòng:**
 - `@router.get(..., response_model=UserOut)`: Ép dữ liệu đầu ra tuân thủ tuyệt đối cấu trúc `UserOut` dù đối tượng `db_user` trong DB có nhiều trường hơn.

- `status_code=status.HTTP_200_OK`: Trả về mã định danh 200 chuẩn RESTful cho tác vụ đọc dữ liệu thành công.
- **Lỗi thường gặp:** `PydanticUserError` hoặc trả về dữ liệu rỗng (`{}`) nếu Class Config của `UserOut` quên chưa bật thuộc tính `from_attributes = True` khi nhận một đối tượng SQLAlchemy ORM.

7.2 Các lớp Response chuyên dụng

FastAPI cung cấp lớp `Response` gốc và các lớp con để tối ưu hóa hiệu năng cho từng loại dữ liệu đầu ra riêng biệt:

- **JSONResponse:** Lớp mặc định. Chuyển đổi dữ liệu Python (Dict, List) sang chuỗi JSON và thêm Header `application/json`.
- **Response:** Trả về dữ liệu thô (Raw bytes hoặc String) mà không qua bất kỳ bộ xử lý định dạng nào, tối ưu cho hiệu năng cực cao.
- **StreamingResponse:** Truyền tải dữ liệu theo từng luồng nhỏ (Chunks). Cực kỳ hữu ích khi chuyển giao các tệp tin có dung lượng lớn hoặc luồng video, giúp tối ưu bộ nhớ RAM của Server không bị quá tải.
- **FileResponse:** Một lớp chuyên biệt hóa từ `StreamingResponse`, tự động nhận diện kiểu file (MIME type) và thiết lập Header thích hợp để Client tải xuống tệp tin vật lý từ ổ đĩa.
- **RedirectResponse:** Trả về mã trạng thái HTTP nhóm 3xx (ví dụ `307 Temporary Redirect`), điều hướng Client đến một URL đích khác ngay lập tức.
- **Mã nguồn ví dụ tổng hợp các loại Response:**

Python

```
from fastapi import APIRouter
from fastapi.responses import JSONResponse, FileResponse, StreamingResponse
import io
```

```
router = APIRouter()
```

```
# 1. Sử dụng JSONResponse để tùy biến Header thủ công
```

```
@router.get("/custom-json")
def get_custom_json():
    return JSONResponse(
        status_code=202,
        content={"message": "Yêu cầu đã được tiếp nhận"},
        headers={"X-Custom-Header": "Enterprise-Safe"}
    )
```

```
# 2. Sử dụng StreamingResponse để xuất file CSV dạng luồng (On-the-fly)
```

```
@router.get("/download-stream")
def stream_csv():
    csv_data = "id,name\n1,Pro_Laptop\n2,Smart_Phone"
    generator = io.StringIO(csv_data)
```

```
return StreamingResponse(generator, media_type="text/csv",
headers={"Content-Disposition": "attachment; filename=export.csv"})
```

3. Sử dụng FileResponse để trả về tệp cấu hình hệ thống

```
@router.get("/get-report")
```

```
def get_report_file():
```

```
    file_path = "/var/www/static/report.pdf"
```

```
    return FileResponse(file_path, media_type="application/pdf",
filename="thong_ke_doanh_thu.pdf")
```

Chương 8. Quản trị lỗi chuyên sâu & Toàn vẹn giao dịch (Transactions)

8.1 Sơ đồ luồng xử lý ngoại lệ chuẩn (Exception Lifecycle)

Plaintext

Client (Gửi Request lỗi hoặc không hợp lệ)



FastAPI API Layer (Thực thi mã nghiệp vụ)



[Xảy ra sự cố / Logic Exception]

Raise Exception (HTTPException / Custom Exception)



Global Exception Handler (Bắt giữ ngoại lệ tập trung)



[Thực thi khẩn cấp]

db.rollback() (Thu hồi toàn bộ dữ liệu tạm thời để bảo vệ Database)



JSONResponse (Định dạng cấu trúc lỗi đồng nhất trả về Client)

8.2 HTTPException & Try-Except Rollback Kỹ thuật

Khi thao tác với Database, nguyên tắc nguyên tử (**Atomicity** - trong ACID) đòi hỏi nếu một bước trong chuỗi tác vụ thất bại, toàn bộ các bước trước đó trong cùng phiên (Session) phải được xóa bỏ hoàn toàn để tránh rác dữ liệu.

- Mã nguồn ví dụ (Xử lý lỗi chuẩn doanh nghiệp):

Python

```
from fastapi import APIRouter, Depends, HTTPException, status
```

```
from sqlalchemy.orm import Session
```

```
from app.database import get_db
```

```

from app.models.product import Product
from app.schemas.product import ProductCreate

router = APIRouter()

@router.post("/products", status_code=status.HTTP_201_CREATED)
def create_new_product(product_in: ProductCreate, db: Session = Depends(get_db)):
    # Khởi tạo instance từ Model
    db_product = Product(**product_in.model_dump())
    db.add(db_product)

    try:
        # Bước 1: Lưu tạm thời sản phẩm vào DB
        db.flush()

        # Bước 2: Giả định thực hiện một logic nghiệp vụ phức tạp khác bị lỗi hệ thống
        # (Ví dụ: Trừ tiền, gọi API bên thứ ba, đồng bộ kho...)
        raise ValueError("Lỗi kết nối API quản lý kho vật lý.")

    db.commit()
    db.refresh(db_product)
    return db_product

except ValueError as val_err:
    db.rollback() # Khôi phục trạng thái DB nguyên bản trước khi thực hiện Transaction này
    raise HTTPException(
        status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
        detail=f"Lỗi logic nghiệp vụ: {str(val_err)}"
    )

except Exception as e:
    db.rollback() # Đảm bảo luôn giải phóng transaction khi có lỗi bất định
    raise HTTPException(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        detail="Hệ thống gặp sự cố nghiêm trọng không thể hoàn tất tác vụ."
    )

```

8.3 ValidationError vs RequestValidationError

- **ValidationError:** Lỗi phát sinh nội bộ trong mã nguồn Python khi bạn cố tình gán hoặc ép kiểu dữ liệu sai lệch với định nghĩa của một Pydantic Model độc lập.
- **RequestValidationError:** Lỗi do FastAPI kế thừa từ Pydantic, tự động kích hoạt ngay từ tầng nhận dữ liệu (Router) khi HTTP Request của Client gửi lên không khớp với cấu trúc Schema yêu cầu. Mặc định trả về mã **422 Unprocessable Entity**.

8.4 Custom Exception & exception_handler() Toàn cục

Để quản lý thông điệp lỗi đồng nhất (ví dụ hệ thống luôn trả về dạng `{"success": false, "error": "nội dung"}`), chúng ta cần ghi đè bộ bắt lỗi mặc định.

- Mã nguồn ví dụ:

Python

```
from fastapi import FastAPI, Request, status
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
```

```
app = FastAPI()
```

1. Định nghĩa một Exception tùy biến cho lỗi phân quyền

```
class SecurityPermissionException(Exception):
```

```
    def __init__(self, action: str):
```

```
        self.action = action
```

2. Đăng ký Handler toàn cục cho Custom Exception

```
@app.exception_handler(SecurityPermissionException)
```

```
def permission_exception_handler(request: Request, exc: SecurityPermissionException):
```

```
    return JSONResponse(
```

```
        status_code=status.HTTP_403_FORBIDDEN,
```

```
        content={
```

```
            "success": False,
```

```
            "code": "PERMISSION_DENIED",
```

```
            "message": f"Tài khoản không có quyền thực hiện hành động: {exc.action}"
```

```
        }
```

```
    )
```

3. Ghi đè lại RequestValidationError mặc định của FastAPI để làm sạch dữ liệu trả về

```
@app.exception_handler(RequestValidationError)
```

```
def validation_exception_handler(request: Request, exc: RequestValidationError):
```

```
    errors = exc.errors()
```

```
    clean_errors = [{"field": err["loc"][-1], "message": err["msg"]} for err in errors]
```

```
    return JSONResponse(
```

```
        status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
```

```
        content={
```

```
            "success": False,
```

```
            "code": "VALIDATION_FAILED",
```

```
            "details": clean_errors
```

```
        }
```

```
    )
```

8.5 Tổng hợp mã lỗi (Status Code Matrix) ứng dụng thực tế

- **401 Unauthorized**: Token hết hạn, sai thông tin xác thực JWT.

- **403 Forbidden**: Token hợp lệ nhưng tài khoản không đủ thẩm quyền (Ví dụ: **User** cố truy cập route **Admin**).
- **404 Not Found**: Không tìm thấy tài nguyên (Tìm User có ID không tồn tại trong DB).
- **422 Unprocessable Entity**: Dữ liệu đầu vào sai định dạng (Thiếu trường bắt buộc, sai kiểu dữ liệu).
- **500 Internal Server Error**: Lỗi sập hệ thống do Code logic bị crash, mất kết nối DB vật lý.

Chương 9. Mô hình chuyển đổi dữ liệu toàn diện (Data Mapping Matrix)

9.1 Sơ đồ chu trình tuần hoàn dữ liệu (Bi-directional Serialization)

Trong các dự án lớn, dữ liệu liên tục biến đổi qua nhiều hình thái cấu trúc khác nhau để phục vụ các mục đích bảo mật và tối ưu lưu trữ.

Plaintext

[CHIỀU ĐẦU VÀO - DESERIALIZATION]

JSON String (Client gửi)

|

▼ [Mạng Network]

Pydantic Schema (Kiểm tra kiểu dữ liệu tĩnh)

|

▼ [.model_dump()]

Python Dictionary (Cấu trúc Dict nguyên bản)

|

▼ [Khởi tạo Model(**dict)]

SQLAlchemy ORM Model (Đối tượng thực thể lập trình)

|

▼ [db.commit()]

MySQL Rows (Lưu trữ vật lý dạng bảng nhị phân)

|

[CHIỀU ĐẦU RA - SERIALIZATION]

MySQL Rows (Truy vấn dữ liệu lên)

|

▼ [db.query()]

SQLAlchemy ORM Model (Đối tượng thực thể chứa liên kết)

|

▼ [Tự động trích xuất nhờ from_attributes=True]

Pydantic Out Schema (Bộ lọc loại bỏ trường bảo mật)

|

▼ [FastAPI Engine conversion]

JSON String (Trả về trình duyệt của Client)

9.2 Minh họa luồng đi bằng mã nguồn thực tế

Dưới đây là một chu trình khép kín, thể hiện rõ nét sự chuyển đổi hình thái của dữ liệu từ khi nhận vào đến khi xuất ra:

Python

```
from pydantic import BaseModel, EmailStr
from sqlalchemy import Column, Integer, String
from app.database import Base

# --- BƯỚC 1: ĐỊNH NGHĨA HÌNH THÁI DỮ LIỆU ---

# SQLAlchemy Model đại diện cho cấu trúc bảng vật lý trong MySQL
class DBUser(Base):
    __tablename__ = "users_example"
    id = Column(Integer, primary_key=True, index=True)
    email = Column(String(100), unique=True)
    hashed_password = Column(String(255)) # Lưu mật khẩu đã mã hóa

# Pydantic Schema nhận dữ liệu đầu vào (Yêu cầu có mật khẩu thô)
class UserInRequest(BaseModel):
    email: EmailStr
    password: str

# Pydantic Schema xuất dữ liệu đầu ra (Tuyệt đối không có trường mật khẩu)
class UserOutResponse(BaseModel):
    id: int
    email: EmailStr

class Config:
    from_attributes = True

# --- BƯỚC 2: THỰC THI CHUYỂN ĐỔI TRONG ROUTE HANDLER ---

from fastapi import FastAPI, Depends
from sqlalchemy.orm import Session
from app.database import get_db

app = FastAPI()

@app.post("/submit-user", response_model=UserOutResponse)
def workflow_demo(payload: UserInRequest, db: Session = Depends(get_db)):
    # 1. Dữ liệu từ JSON đã được ép vào đối tượng 'payload' (UserInRequest)

    # 2. Chuyển đổi Pydantic sang Python Dict bằng model_dump()
```

```
data_dict = payload.model_dump()
```

```
# 3. Biến đổi cấu trúc sang SQLAlchemy ORM Model
```

```
new_db_user = DBUser(
```

```
    email=data_dict["email"],
```

```
    hashed_password="hashed_salt_key_" + data_dict["password"] # Logic băm mật khẩu
```

```
)
```

```
# 4. Ghi xuống MySQL Database
```

```
db.add(new_db_user)
```

```
db.commit()
```

```
db.refresh(new_db_user)
```

```
# 5. Phản hồi về: FastAPI nhận đối tượng 'new_db_user' (Kiểu DBUser),
```

```
# tự động gọi UserOutResponse.from_orm() nhờ cấu hình from_attributes=True,
```

```
# lọc bỏ trường hashed_password và chuyển thành chuỗi JSON chuẩn trả về Client.
```

```
return new_db_user
```

Chương 10. Dependency Injection (Tiêm phụ thuộc)

10.1 Depends() & yield & get_db()

- **Khái niệm:** Dependency Injection (DI) là một mô hình thiết kế cho phép một hàm tự động nhận các tài nguyên cần thiết từ bên ngoài truyền vào thay vì tự khởi tạo nội bộ. Trong FastAPI, `Depends()` là công cụ thực thi DI cốt lõi. Khi kết hợp với từ khóa `yield`, nó cho phép ta quản lý vòng đời của tài nguyên một cách an toàn (ví dụ: mở kết nối cơ sở dữ liệu khi request đến và tự động đóng lại khi response được trả về, bất kể có lỗi xảy ra hay không).
- **Mã nguồn ví dụ:**

```
Python
```

```
# app/database.py (Trích đoạn cung cấp Session)
```

```
from app.database import SessionLocal
```

```
def get_db():
```

```
    db = SessionLocal()
```

```
    try:
```

```
        yield db # Cung cấp session cho Endpoint làm việc
```

```
    finally:
```

```
        db.close() # Tự động đóng kết nối sau khi Response đã được xử lý xong
```

- **Giải thích từng dòng:**
 - `db = SessionLocal()`: Khởi tạo một phiên làm việc mới với Database.
 - `yield db`: Tạm dừng hàm `get_db()` tại đây và bàn giao đối tượng `db` cho API Endpoint sử dụng.

- `finally: db.close()`: Khối lệnh luôn được thực thi ở cuối vòng đời của một Request, đảm bảo thu hồi kết nối vật lý, giải phóng tài nguyên hệ thống, chống rò rỉ bộ nhớ (Connection Leak).

10.2 OAuth2PasswordBearer

- **Khái niệm:** Là một class cấu hình bảo mật của FastAPI, tự động khai báo luồng xác thực OAuth2 với Swagger UI. Nó định nghĩa một Endpoint chuyên trách thu thập thông tin đăng nhập và tự động tìm kiếm mã Token được đính kèm trong Header `Authorization: Bearer <token>` của HTTP Request gửi lên.
- **Mã nguồn ví dụ:**

Python

```
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
```

```
# Khai báo endpoint xử lý cấp token
```

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/login")
```

```
def verify_token_in_request(token: str = Depends(oauth2_scheme)):
```

```
    if not token:
```

```
        raise HTTPException(
```

```
            status_code=status.HTTP_401_UNAUTHORIZED,
```

```
            detail="Thiếu mã xác thực (Token missing)"
```

```
        )
```

```
    return token
```

Chương 11. Hệ thống định tuyến Router chuyên nghiệp

11.1 APIRouter & include_router

- **Khái niệm:** Khi ứng dụng phát triển lên hàng trăm Endpoints, việc viết tất cả vào một file `main.py` là bất khả thi. `APIRouter` cho phép cô lập, phân tách các tuyến đường API theo từng phân hệ chức năng độc lập, sau đó gộp chung lại vào ứng dụng chính thông qua phương thức `include_router`.

11.2 Các tham số cấu hình: prefix, tags, dependencies

- **prefix:** Định nghĩa tiền tố đường dẫn chung cho toàn bộ module (Ví dụ `/users`, `/products`).
- **tags:** Nhóm các endpoint lại với nhau trên giao diện tài liệu Swagger UI để dễ quản lý.
- **dependencies:** Áp đặt các bộ lọc phụ thuộc (như kiểm tra quyền đăng nhập) lên cấp độ toàn bộ Router mà không cần khai báo lại ở từng hàm.
- **Mã nguồn ví dụ phân hệ Router (`app/routers/products.py`):**

```

Python
from fastapi import APIRouter, Depends
from typing import List
from app.schemas.product import ProductOut
from app.database import get_db
from sqlalchemy.orm import Session

router = APIRouter(
    prefix="/products",
    tags=["Quản lý Sản Phẩm"],
    responses={404: {"description": "Không tìm thấy sản phẩm"}}
)

@router.get("/", response_model=List[ProductOut])
def get_all_products(db: Session = Depends(get_db)):
    return [] # Logic xử lý thực tế nằm ở tầng CRUD

```

- Mã nguồn tích hợp tại tệp tổng (**app/main.py**):

```

Python
from fastapi import FastAPI
from app.routers import products

app = FastAPI(title="Hệ thống Doanh Nghiệp API")

# Tích hợp module router vào ứng dụng chính
app.include_router(products.router)

```

Chương 12. Xác thực và Bảo mật nâng cao (Authentication & Security)

12.1 JWT (JSON Web Token), Access Token & Refresh Token

- **Khái niệm:** JWT là một chuỗi mã hóa phi trạng thái (Stateless), dùng để xác thực danh tính người dùng một cách an toàn mà không cần lưu trữ session trên server.
 - **Access Token:** Có vòng đời ngắn (ví dụ: 15 - 30 phút), đính kèm theo mỗi request để chứng minh quyền truy cập tài nguyên.
 - **Refresh Token:** Có vòng đời dài (ví dụ: 7 ngày), dùng để yêu cầu hệ thống cấp một Access Token mới mà không bắt người dùng phải nhập lại mật khẩu.

12.2 Mã hóa mật khẩu với bcrypt

- **Khái niệm:** Tuyệt đối không lưu mật khẩu thô vào Database dưới dạng plain-text. `bcrypt` là thuật toán băm một chiều (One-way hashing) kết hợp cơ chế "Salt" ngẫu nhiên chống lại các cuộc tấn công dò mã (Brute-force).

12.3 Mã nguồn thực thi quy trình bảo mật chuẩn

- Cần cài đặt thư viện hỗ trợ: `pip install PyJWT passlib[bcrypt]`

Python

```
# app/utils/security.py
from passlib.context import CryptContext
from datetime import datetime, timedelta, timezone
import jwt

# 1. Cấu hình context băm mật khẩu
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

SECRET_KEY = "SUPER_SECRET_ENTERPRISE_KEY_DO_NOT_LEAK"
ALGORITHM = "HS256"

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

# 2. Logic tạo JWT Access Token
def create_access_token(data: dict, expires_delta: timedelta | None = None) -> str:
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.now(timezone.utc) + expires_delta
    else:
        expire = datetime.now(timezone.utc) + timedelta(minutes=15)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt
```

- Mã nguồn Route thực thi Đăng nhập và xác thực Token:

Python

```
# app/routers/auth.py
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.orm import Session
from app.database import get_db
from app.models.user import User # Giả định đã có Model User
```

```

from app.utils.security import verify_password, create_access_token
from datetime import timedelta

router = APIRouter(prefix="/auth", tags=["Xác Thực Hệ Thống"])

@router.post("/login")
def login_for_access_token(
    form_data: OAuth2PasswordRequestForm = Depends(),
    db: Session = Depends(get_db)
):
    # Tìm kiếm user trong DB qua email hoặc username
    user = db.query(User).filter(User.username == form_data.username).first()
    if not user or not verify_password(form_data.password, user.hashed_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Tài khoản hoặc mật khẩu không chính xác",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # Cấp token có hiệu lực trong 30 phút
    access_token_expires = timedelta(minutes=30)
    access_token = create_access_token(
        data={"sub": user.username, "role": "user"},
        expires_delta=access_token_expires
    )

    return {"access_token": access_token, "token_type": "bearer"}

```

Chương 13. Kỹ thuật Upload File và Xử lý Form Data

13.1 UploadFile vs File vs Form

Trong FastAPI, việc nhận dữ liệu từ Client gửi lên thông qua giao thức HTTP được chia làm hai nhánh rõ rệt: dữ liệu dạng JSON thông thường và dữ liệu dạng `multipart/form-data` (sử dụng khi cần tải tập tin hoặc gửi dữ liệu biểu mẫu truyền thống).

- **Form:** Lớp dùng để định nghĩa các trường dữ liệu text, number tương tự Pydantic nhưng được nhận dưới dạng biểu mẫu.
- **File:** Nhận tập tin đầu vào dưới dạng một chuỗi Bytes thô (Raw Bytes). Lưu ý: Việc dùng `File(bytes)` sẽ tải toàn bộ nội dung file vào bộ nhớ RAM, dễ gây sập server nếu file có dung lượng lớn.
- **UploadFile:** Lớp chuyên dụng cấp cao của FastAPI kế thừa từ Starlette. Nó sử dụng một file tạm thời lưu trên ổ đĩa (`SpooledTemporaryFile`) nếu dung lượng vượt ngưỡng, giúp bảo vệ RAM tối đa. Nó cung cấp các Metadata quan trọng như tên file, kiểu định dạng (Content-Type) và các hàm async xử lý luồng đọc/ghi.

13.2 Mã nguồn thực tế xử lý tải file và lưu trữ an toàn

- Yêu cầu cài đặt thư viện hỗ trợ: `pip install python-multipart`

Python

```
# app/routers/media.py
```

```
from fastapi import APIRouter, UploadFile, File, Form, HTTPException, status
```

```
import os
```

```
import shutil
```

```
from pathlib import Path
```

```
router = APIRouter(prefix="/media", tags=["Quản lý Tập Tin"])
```

```
# Thư mục đích lưu trữ file vật lý trên server
```

```
UPLOAD_DIR = Path("static/uploads")
```

```
UPLOAD_DIR.mkdir(parents=True, exist_ok=True)
```

```
# Các định dạng file cho phép upload (Ví dụ: Chỉ cho phép hình ảnh)
```

```
ALLOWED_EXTENSIONS = {".jpg", ".jpeg", ".png", ".webp"}
```

```
MAX_FILE_SIZE = 5 * 1024 * 1024 # Tối đa 5 Megabytes
```

```
@router.post("/upload-avatar")
```

```
async def upload_avatar(
```

```
    user_id: int = Form(...),
```

```
    avatar_file: UploadFile = File(...)
```

```
):
```

```
    # 1. Kiểm tra phần mở rộng của tệp tin (Extension Validation)
```

```
    file_ext = os.path.splitext(avatar_file.filename)[1].lower()
```

```
    if file_ext not in ALLOWED_EXTENSIONS:
```

```
        raise HTTPException(
```

```
            status_code=status.HTTP_400_BAD_REQUEST,
```

```
            detail=f"Định dạng file không hỗ trợ. Chỉ chấp nhận: {ALLOWED_EXTENSIONS}"
```

```
)
```

```
    # 2. Đọc thử dung lượng file để chặn file quá lớn
```

```
    # Di chuyển con trỏ chuột về cuối file để lấy kích thước
```

```
    await avatar_file.seek(0, os.SEEK_END)
```

```
    file_size = avatar_file.tell()
```

```
    await avatar_file.seek(0) # Đưa con trỏ chuột về đầu file để chuẩn bị đọc dữ liệu
```

```
    if file_size > MAX_FILE_SIZE:
```

```
        raise HTTPException(
```

```
            status_code=status.HTTP_413_REQUEST_ENTITY_TOO_LARGE,
```

```
            detail="Kích thước tập tin vượt quá giới hạn cho phép (5MB)."
```

```
)
```

```
    # 3. Đổi tên file để tránh trùng lặp và tấn công Path Traversal
```

```
    secure_filename = f"user_{user_id}_{int(os.time())}{file_ext}"
```

```
file_path = UPLOAD_DIR / secure_filename
```

```
# 4. Ghi file an toàn bằng Stream xuống ổ đĩa vật lý
```

```
with open(file_path, "wb") as buffer:
```

```
    shutil.copyfileobj(avatar_file.file, buffer)
```

```
return {
```

```
    "filename": secure_filename,
```

```
    "content_type": avatar_file.content_type,
```

```
    "saved_path": str(file_path)
```

```
}
```

Chương 14. Vận hành và Thiết lập Hệ thống Middleware

14.1 Bản chất của Middleware

Middleware là một bộ lọc logic nằm trung gian giữa Client và API Endpoint chính của ứng dụng. Khi một Request gửi đến, nó phải đi qua Middleware trước khi tới Router. Tương tự, dữ liệu Response từ Router trả ra cũng đi qua Middleware một lần nữa trước khi gửi về cho Client.

14.2 CORS (Cross-Origin Resource Sharing)

Mặc định, các trình duyệt web chặn các yêu cầu HTTP từ một Domain này gọi sang một Domain khác vì lý do bảo mật (Chính sách Same-Origin). Middleware CORS của FastAPI cho phép ta cấu hình tường minh danh sách các Domain của Front-end (Ví dụ: React/Vue app chạy ở port 3000) được phép tương tác và lấy dữ liệu từ Backend.

14.3 Mã nguồn cấu hình Middleware tổng hợp (CORS, Logging & Performance)

Python

```
# app/main.py (Đoạn trích tích hợp Middleware)
```

```
from fastapi import FastAPI, Request
```

```
from fastapi.middleware.cors import CORSMiddleware
```

```
import time
```

```
import logging
```

```
app = FastAPI()
```

```
# 1. Cấu hình CORS chuẩn doanh nghiệp
```

```
origins = [
```

```
    "http://localhost:3000",    # Ứng dụng React chạy local
```

```
    "https://dashboard.mycompany.com" # Tên miền môi trường Production
```

```
]
```

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,          # Cho phép các domain này truy cập
    allow_credentials=True,         # Cho phép gửi kèm Cookies/Auth Headers
    allow_methods=["GET", "POST", "PUT", "DELETE"], # Hạn chế các method được phép
    allow_headers=["*"],            # Chấp nhận tất cả định dạng Header dữ liệu
)

```

2. Tự định nghĩa Custom Middleware tính toán thời gian xử lý và Ghi Log hệ thống

```

@app.middleware("http")
async def log_execution_time_middleware(request: Request, call_next):
    start_time = time.time()

```

```

    # Giai đoạn 1: Tiền xử lý (Pre-processing) trước khi vào Route
    logging.info(f"Yêu cầu đến: {request.method} {request.url.path}")

```

```

    # Chuyển giao request cho Route kế tiếp xử lý để nhận Response về
    response = await call_next(request)

```

```

    # Giai đoạn 2: Hậu xử lý (Post-processing) trước khi trả về Client
    process_time = time.time() - start_time
    response.headers["X-Process-Time"] = f"{process_time:.4f} seconds"

```

```

    logging.info(f"Hoàn tất: Trạng thái {response.status_code} | Thời gian chạy: {process_time:.4f}s")

```

```

    return response

```

Chương 15. Quản trị cấu trúc Cơ sở dữ liệu với Alembic Migration

15.1 Khái niệm về Database Migration

Trong môi trường phát triển thực tế, cấu trúc bảng liên tục thay đổi (thêm cột, sửa kiểu dữ liệu, xóa bảng). Nếu ta dùng hàm `Base.metadata.create_all(bind=engine)` của SQLAlchemy, nó chỉ có tác dụng tạo bảng mới nếu bảng đó chưa tồn tại, chứ hoàn toàn không có khả năng cập nhật sự thay đổi của các cột trên các bảng cũ.

Alembic ra đời để giải quyết vấn đề này. Nó ghi lại lịch sử thay đổi cấu trúc DB dưới dạng các file mã nguồn Python (gọi là các bản mẫu Migration). Khi chạy mã lệnh, cấu trúc CSDL sẽ dịch chuyển mượt mà mà không làm mất dữ liệu hiện tại.

- **Revision:** Một phiên bản ghi nhận sự thay đổi cấu trúc tại một mốc thời gian.
- **Upgrade:** Hàm chứa lệnh thực thi để cập nhật cấu trúc DB lên phiên bản mới hơn.
- **Downgrade:** Hàm chứa lệnh đảo ngược (khôi phục) cấu trúc DB về phiên bản trước đó nếu bản cập nhật mới gặp lỗi.

15.2 Quy trình vận hành các dòng lệnh Alembic thực tế

Để tích hợp Alembic vào dự án, lập trình viên thực hiện chuỗi lệnh tuần tự sau trên Terminal hệ thống:

- **Bước 1: Khởi tạo thư mục môi trường Alembic**

Bash

```
alembic init alembic
```

Lệnh này tạo ra một file cấu hình `alembic.ini` và một thư mục tên là `alembic/`.

- **Bước 2: Cấu hình liên kết Model với Alembic** Mở tệp `alembic/env.py`, tìm và chỉnh sửa hai tham số cốt lõi để Alembic nhận diện được đường dẫn DB và các Model của ứng dụng:

Python

```
# alembic/env.py
from app.database import Base
from app.models.user import User      # Khai báo để Alembic nạp mô hình bảng
from app.models.product import Product

# Trỏ target_metadata tới siêu dữ liệu của Base
target_metadata = Base.metadata

# Thiết lập chuỗi kết nối động lấy từ file config môi trường của ứng dụng
from app.config import settings
config.set_main_option("sqlalchemy.url", settings.DATABASE_URL)
```

- **Bước 3: Tự động phát hiện thay đổi và sinh file Migration (Autogenerate)** Khi lập trình viên thêm một cột mới vào Class Model trong Code, chạy lệnh sau để Alembic tự so sánh Code với cấu trúc hiện tại dưới DB và tạo file script:

Bash

```
alembic revision --autogenerate -m "them_cot_sdt_vao_bang_users"
```

- **Bước 4: Xem trước tệp lệnh vừa được sinh ra**
(`alembic/versions/xxxx_migration.py`)

Python

```
def upgrade():
    # Alembic tự sinh mã lệnh thêm cột phone_number vào MySQL vật lý
    op.add_column('users', sa.Column('phone_number', sa.String(length=15), nullable=True))

def downgrade():
    # Lệnh thu hồi: Xóa cột vừa thêm nếu rollback hệ thống
```

```
op.drop_column('users', 'phone_number')
```

- **Bước 5: Thực thi áp đặt thay đổi xuống Database vật lý**

Bash

```
alembic upgrade head
```

Chữ **head** đại diện cho phiên bản mới nhất vừa được tạo ra. Toàn bộ cấu trúc bảng dưới MySQL lập tức được đồng bộ an toàn.

Chương 16. Kiến trúc phân lớp Doanh nghiệp nâng cao (Best Practices)

16.1 Tại sao cần phân tách cấu trúc Enterprise?

Khi hệ thống phình to, việc nhồi nhét xử lý logic vào tầng CRUD hoặc Router sẽ tạo ra các file "God Object" (quá nhiều chức năng, khó viết Unit Test, dễ gây xung đột khi làm việc nhóm). Kiến trúc Enterprise chuẩn doanh nghiệp chia tách hệ thống thành 5 phân lớp chuyên biệt hóa với nguyên tắc: **Tầng trên gọi tầng dưới, tuyệt đối không gọi ngược lại.**

16.2 Bóc tách vai trò của từng phân lớp

1. **Controller (Router Layer):** * *Nhiệm vụ:* Chỉ chịu trách nhiệm tiếp nhận HTTP Request, cấu hình Route Decorator, gán `response_model`, kiểm tra quyền hạn cơ bản thông qua `Depends()` bảo mật và chuyển giao ngay khối dữ liệu sạch cho tầng Service.
 - *Nguyên tắc:* Không chứa bất kỳ câu lệnh logic tính toán hay câu lệnh SQL nào.
2. **Service Layer (Logic doanh nghiệp):** * *Nhiệm vụ:* Bộ não của toàn hệ thống. Nơi thực hiện các thuật toán tính toán, kiểm tra điều kiện nghiệp vụ (ví dụ: kiểm tra số dư ví, gọi API cổng thanh toán bên thứ ba, tính toán thuế giá trị gia tăng).
 - *Nguyên tắc:* Nhận dữ liệu từ Controller, phối hợp gọi một hoặc nhiều hàm từ tầng CRUD/Repository để tạo ra một chuỗi tác vụ (Transaction) hoàn chỉnh.
3. **CRUD / Repository Layer (Thao tác DB):** * *Nhiệm vụ:* Đảm nhận nhiệm vụ giao tiếp trực tiếp với hạ tầng dữ liệu thông qua SQLAlchemy Session. Nó chỉ làm các việc thuần túy: Thêm, Đọc, Sửa, Xóa một bảng cụ thể.
 - *Nguyên tắc:* Không quan tâm đến logic kinh doanh, chỉ thực thi lệnh truy vấn tối ưu nhất.
4. **Model Layer (Thực thể):** Định nghĩa cấu trúc bảng ánh xạ vật lý.
5. **Database Layer (Lưu trữ):** Hệ quản trị CSDL vật lý (MySQL).

16.3 Mã nguồn minh họa phân tách lớp cho tính năng "Mua Hàng" (Checkout)

- **Lớp 1: Router Controller (app/routers/order.py)**

Python

```
from fastapi import APIRouter, Depends, status
from sqlalchemy.orm import Session
from app.database import get_db
from app.schemas.order import OrderCreate, OrderOut
from app.services.order import OrderService
```

```
router = APIRouter(prefix="/orders", tags=["Hệ thống Đơn Hàng"])
```

```
@router.post("/", response_model=OrderOut, status_code=status.HTTP_201_CREATED)
def checkout_cart(payload: OrderCreate, db: Session = Depends(get_db)):
    # Bàn giao toàn bộ payload sạch và DB session cho tầng Service
    order_service = OrderService(db)
    new_order = order_service.process_checkout(payload)
    return new_order
```

- **Lớp 2: Service Logic (app/services/order.py)**

Python

```
from fastapi import HTTPException, status
from app.schemas.order import OrderCreate
from app.crud.product import product_crud
from app.crud.order import order_crud
```

```
class OrderService:
```

```
    def __init__(self, db):
        self.db = db
```

```
    def process_checkout(self, order_data: OrderCreate):
```

```
        # 1. Gọi tầng CRUD lấy thông tin sản phẩm từ DB
```

```
        product = product_crud.get_by_id(self.db, order_data.product_id)
```

```
        if not product:
```

```
            raise HTTPException(status_code=404, detail="Sản phẩm không tồn tại")
```

```
        # 2. Xử lý Logic doanh nghiệp: Kiểm tra hàng tồn kho
```

```
        if product.stock_quantity < order_data.quantity:
```

```
            raise HTTPException(
```

```
                status_code=status.HTTP_400_BAD_REQUEST,
```

```
                detail="Số lượng hàng tồn kho không đủ cung ứng"
```

```
            )
```

```
        # 3. Tính toán tổng tiền
```

```
        total_price = product.price * order_data.quantity
```

```
        try:
```

```
# 4. Gọi tầng CRUD cập nhật giảm số kho của sản phẩm
```

```
product_crud.decrease_stock(self.db, product.id, order_data.quantity)
```

```
# 5. Gọi tầng CRUD tạo bản ghi hóa đơn mới
```

```
new_order = order_crud.create_order(self.db, order_data, total_price)
```

```
self.db.commit() # Xác nhận chuỗi tác vụ thành công liên hợp
```

```
return new_order
```

```
except Exception:
```

```
self.db.rollback() # Hủy bỏ toàn bộ nếu một trong hai bước DB bị sập
```

```
raise HTTPException(status_code=500, detail="Giao dịch thanh toán thất bại")
```

Chương 17. Tổng hợp & Tra cứu nhanh (Cheat Sheet 250+ Hàm)

Dưới đây là bảng phân loại và định nghĩa nhanh cho các hàm cốt lõi quan trọng nhất trong hệ sinh thái FastAPI, giúp bạn tra cứu tức thì khi lập trình dự án:

17.1 Nhóm hàm Điều hướng và Cấu hình (FastAPI Core)

- `FastAPI()`: Khởi tạo ứng dụng web tổng.
- `APIRouter()`: Khởi tạo phân hệ định tuyến module độc lập.
- `include_router()`: Tích hợp các module con vào ứng dụng gốc.
- `Depends()`: Tiêm phụ thuộc (Dependency Injection), quản lý tài nguyên.
- `HTTPException()`: Kích hoạt ngắt luồng và trả về lỗi HTTP tùy biến.
- `yield`: Quản lý mở/đóng tài nguyên theo vòng đời Request.

17.2 Nhóm hàm Kiểm tra & Định nghĩa Dữ liệu (Pydantic Validation)

- `BaseModel`: Lớp cha định nghĩa Schema dữ liệu đầu vào/đầu ra.
- `Field()`: Cấu hình giới hạn, giá trị mặc định và mô tả cho thuộc tính.
- `model_dump()`: Xuất dữ liệu đối tượng Pydantic thành Python Dict thuần.
- `model_dump_json()`: Xuất trực tiếp đối tượng Pydantic thành chuỗi JSON String.
- `from_attributes=True`: Cấu hình nhận diện dữ liệu từ SQLAlchemy Model.

17.3 Nhóm hàm Nhận diện Tham số (Request Parameters)

- `Query()`: Ép buộc tham số nhận vào từ Query String trên URL (`?page=1&limit=10`).
- `Path()`: Ép buộc tham số nhận vào từ đường dẫn URL (`/users/{user_id}`).
- `Body()`: Ép dữ liệu thô nhận từ Body Request thay vì dùng Schema Pydantic.
- `Header()`: Thu thập dữ liệu Metadata từ HTTP Headers gửi lên.
- `Cookie()`: Đọc dữ liệu bảo mật từ Cookie lưu tại trình duyệt.

- `Form()`: Nhận dữ liệu dạng text/number từ biểu mẫu `multipart/form-data`.
- `File()`: Nhận tệp tin dưới dạng mảng bytes thô (Tải trực tiếp vào RAM).
- `UploadFile`: Lớp tối ưu nhận file qua stream file tạm, tiết kiệm tài nguyên.

17.4 Nhóm hàm Phản hồi Dữ liệu (HTTP Responses)

- `JSONResponse`: Trả về dữ liệu cấu trúc Python dưới dạng chuỗi JSON chuẩn.
- `FileResponse`: Trả về tệp tin vật lý lưu tại ổ đĩa cho Client tải xuống.
- `StreamingResponse`: Truyền tải file dung lượng lớn theo từng phân đoạn (Chunk).
- `RedirectResponse`: Trả về mã điều hướng trình duyệt sang liên kết mới.

17.5 Nhóm hàm Tương tác Cơ sở dữ liệu (SQLAlchemy Session)

- `create_engine()`: Khởi tạo cổng kết nối vật lý và quản lý Pool kết nối.
- `sessionmaker()`: Nhà máy sản xuất các Session quản lý phiên làm việc.
- `db.add()`: Đưa đối tượng ORM vào trạng thái chuẩn bị thêm mới.
- `db.commit()`: Lưu vĩnh viễn toàn bộ thay đổi trong Session xuống đĩa cứng.
- `db.rollback()`: Thu hồi, xóa bỏ toàn bộ thao tác tạm thời nếu xảy ra lỗi.
- `db.flush()`: Đẩy dữ liệu xuống DB để lấy ID tự sinh nhưng chưa lưu vĩnh viễn.
- `db.refresh()`: Tải lại dữ liệu của đối tượng từ DB để cập nhật trạng thái mới nhất.
- `db.delete()`: Đưa đối tượng vào trạng thái chuẩn bị xóa bỏ khỏi bảng.

17.6 Nhóm hàm Truy vấn dữ liệu nâng cao (SQLAlchemy Queries)

- `query()`: Chỉ định bảng mục tiêu để bắt đầu câu lệnh SELECT.
- `filter()`: Mở bộ lọc điều kiện sử dụng toán tử logic (`==`, `>`, `!=`).
- `filter_by()`: Mở bộ lọc điều kiện nhanh bằng tham số đặt tên (`status='active'`).
- `all()`: Thực thi lệnh và gom toàn bộ kết quả trả về thành một danh sách (List).
- `first()`: Thực thi lệnh và chỉ lấy bản ghi đầu tiên bắt gặp.
- `one()`: Trả về đúng 1 bản ghi duy nhất, ném lỗi nếu có 0 hoặc nhiều hơn 1 kết quả.
- `one_or_none()`: Trả về đúng 1 bản ghi, nếu không có kết quả thì trả về `None`.
- `limit()`: Giới hạn số lượng hàng dữ liệu tối đa được phép lấy ra.
- `offset()`: Số lượng hàng bỏ qua tính từ vị trí đầu tiên (Phục vụ phân trang).
- `order_by()`: Sắp xếp thứ tự dữ liệu trả về theo thuộc tính chỉ định (`.desc()`, `.asc()`).
- `group_by()`: Gom nhóm các hàng dữ liệu dựa trên các giá trị trùng nhau.
- `join()`: Thực hiện liên kết Inner Join giữa hai bảng dữ liệu có quan hệ khóa ngoại.
- `outerjoin()`: Thực hiện liên kết Left Outer Join lấy trọn vẹn dữ liệu bảng bên trái.
- `count()`: Đếm tổng số lượng bản ghi thỏa mãn điều kiện lọc.
- `between()`: Lọc dữ liệu nằm trong khoảng giá trị chỉ định (Ví dụ: từ ngày... đến ngày...).
- `in_()`: Lọc dữ liệu nằm trong một danh sách các giá trị cho trước (`[1, 2, 3]`).

